

设计与开发 SKILL 指南编制规范—93 号规范

EOS Design-Development SKILL Guide Authoring Spec

本规范是什么：统一 EOS 设计线上"设计与开发落地"各任务 SKILL 指南的**编制**，此类方案亦称操作规程——总骨架、各章编制规则、跨文档引用与对接，使全链 SKILL 指南读成同一个方法在不同链位的实例化；该方法是 20 号报告的由需求到方案的逐对生成、 $Y=F(X)$ 。SKILL 指南是权威源；其 SKILL 是可执行提取，派生关系见 §5.3。

创建：2026-09-04 | **版本：**第 75 版

一、总则

1.1 目的与定位

设计与开发落地 = 把产品数据链切成相邻的"需求-方案结对"，由需求开发出方案、并由方案迭代需求，正反交替逐个需求-方案对收敛。每一个需求-方案对即一次 $Y=F(X)$ 求值：X 为需求，Y 为方案，F 为规则，即 X 侧/Y 侧树形与节点规则 + Y 承接 X 的承接规则。推进分两步：预处理定义规则、结构化存量、初步承接；正式设计由需求叶节点驱动，走承接设计 → 分解 → 覆盖检查。方法完整定义见 20 号报告。

一份 SKILL 指南就是落地方法在一个任务位上的操作规程：先交代这个任务位在链上的位置——属于哪个"需求-方案对"、哪一阶段；再规定它消费的输入文档、产出的输出文档、两者转换的规则、转换流程与检验规则，以及读写哪些产品数据。

本规范规定此类 SKILL 指南怎么编：章序、各章该写什么、不该写什么、跨文档如何保持一致。它不重述 EOS 领域设计模式与运行时协议，也不重述方法论证——前者引 91 规范，后者引 20 号报告。

1.2 适用范围

- 适用于 **EOS 设计线 SKILL 指南**：biz/eng/nfr 三路径 01→07 链上的承接/分解任务，即 spr2bpl、bpl2bpd、bpd2srb、srb2srl、srl2pa 及 nfr 链对应任务。本规范按**方法级**编制、不绑定 biz/eng/nfr 具体名词，各任务以自己的输入/输出产品数据填充。
- 同一需求-方案对内的设计分三级、按需取用：**框架设计**出框架（根→枝梢）、**概要设计**出概要（根→叶，各级只到说明）、**详细定义**出定义（概要 + 各节点的详细定义）。相邻两级可以合做、也可以省略或裁剪——某份产品数据不必每一级都停。**阶段一可以由两个并列任务分担同一份产品数据的两个视角**，合占一个任务位。

- **任务编号的字母位记"这一步做哪几级"**：a = 框架设计、b = 概要设计、c = 详细定义，连写表示合做（ab = 框架与概要一手做完）；**末位记视角**，r = 职责视角、p = 产品开发视角。任务位号取需求-方案对的序号。
- **叶由上游直接给出的步是特例**：相关方需求 → 业务流程，以及职责视角业务流程概要 → 产品开发视角业务流程概要——这两处的业务文档、任务节点、用例场景三级由上游整段给出，**没有待落之叶**，本任务的活是**组装干枝，并逐级判定这三级各自的落位**。
- **暂不适用 ort00~ort03 需求预处理链**，即原始需求的文本化、切分、澄清、规范化。ort 属 20 号方法的"预处理过程"，但 EOS 已将其独立成链、现行 SKILL 模板不同，维持 91 规范 附录A 既有模板。如需同构再行扩展。

1.3 规范性引用

引用	角色	本规范使用方式
20 号报告：设计与开发落地方法	方法权威：逐对生成 / $Y=F(X)$ / 树的三形态 / 规则 F / 两道关口 / 需求树的来历	方法概念与原理引此，不重复论证
91-eos-biz-eng-spec.md	EOS 领域权威：产品数据文档结构 §11、节点类型与领域设计模式 §1§9、 Skill 运行时协议与格式约定 附录A、各路径任务清单 §7§10	领域规则、schema、运行时协议一律引此
94-eos-sys-dev-dbs.md	树权威 ：各产品数据的树形/级语义，即分解结构—— 各产品数据树成立的理论依据 §1.2、 三个形态与级语义 §1.3、 形态之间的承接与已定的树间对应 §1.4、 R / P 树所指 ，以及 §二 各产品数据的分解结构。R 即职责视角、对应 SL；P 即产品开发视角、对应 PL；下标 0 = 平台本身、 $n \geq 1$ 各业务	产品数据树一律引此，本规范的必给图见 §2.3；数据文件「树画像」区按其对齐，不另设树口径
00-doc-conventions.md	通用文档约定， 对 SKILL 指南为规范性 ——其 §八 表达风格各节全数适用；§十三 只取其 Step 布局总览表的格式与编号约束，§13.4 的 Step 详细操作表不在本类文档使用	文档格式/编号/版本/表达的通用约定；§八 在本类文档上的落地口径与判据见 §2.4，§十三 的引用见 §3.5
各产品数据文件 01~07 自身	各自结构与内容的单源，见 §四.1	结构只引用，不搬运

1.4 术语（简表）

术语	指代
SKILL 指南	EOS 设计线某任务位的操作规程文档，文件名形如 eos- {任务编号} - {转换串} .md；本编号法之前的文件保留原名，更名时按本式； 该任务 SKILL 的权威源 ，SKILL 由它提取，派生关系见 §5.3。91 附录A 第二部分「编写 Skill 文档」讲的是 SKILL.md 自身怎么写，与本词分工不同。本规范的约束对象

术语	指代
输入文档 / X	本任务消费的输入文档，即上游任务的产出或本需求-方案对阶段一的产出；本次映射的 自变量
输出文档 / Y	本任务产出的产品数据文档，本次映射的 函数值
需求-方案对	产品数据链上 相邻两份产品数据 构成的一次"需求→方案"转换——上游为需求、下游为方案；整条链即由相邻的需求-方案对串成，见 §1.1。编号写作需求-方案对①/②/③
框架 / 概要 / 定义	树级语义取自 94-eos-sys-dev-dbs.md §1.3： 框架 = 根→枝梢、概要 = 根→叶（各级只到说明）、定义 = 概要 + 各节点的详细定义 。树级语义一律引此；本规范另取它的一条推论： 三者都不是"需求-方案对"的一端——一个需求-方案对的两端是需求与方案两份产品数据、各取其最深的形态，框架与概要都是方案侧的中间形态，不算一端
边界文档	链上 相邻两任务 的接缝文档—— A 产出它、B 消费它。A = 上游任务、B = 下游任务 ，随接缝而变，不是固定角色
schema	某类产品数据文档的结构，即节点类型/要素清单/字段/状态；全链单源，见 §四.1
方案侧 / 需求侧	边界文档在 A 的第三章「输出文档结构及规则」 的视图，它在那里被生产、带移交承诺；在 B 的第二章「输入文档结构及规则」 的视图，它在那里被消费、带受理前提
接口契约	A 的第三章的移交承诺与 B 的第二章的受理前提互为对侧；并排核对即契约校验
单承接	树侧口径取自 94-eos-sys-dev-dbs.md §1.4： 上游取最深可用原子，即其叶；下游落到干枝或枝梢，以语义定级。基数可 N:1、不可 1:N ；该处并含 两侧理由不同源的说明 。本规范 §3.4 的"各级干枝/枝梢"即该口径的落地说法；方法论论证见 20 号报告
覆盖检查	"承接了的一定被实现"：某节点承接的全部需求语义 $\subseteq$ 其叶节点语义之并

二、SKILL 指南总骨架

2.1 骨架总览

每份系统或构件设计类 SKILL 指南按五主章组织：

章	章名	定位	对应 20 号方法
一	目的与目标	本任务要达成什么、处在链上的什么位置——属哪个"需求-方案对"、哪一阶段	需求-方案对与链定位
二	输入文档结构及规则	X 的 需求侧 视图	X 的树与各级语义

章	章名	定位	对应 20 号方法
三	输出文档结构及规则	Y 的方案侧视图	Y 侧规则 + 分解源判据
四	输入到输出的过程及规则	X→Y 的映射与判定	承接规则，即 F 的第三块
五	执行步骤及检验规则	设计动作序列 + 关口	正式设计过程 + 两道关口

章序逻辑：先文档后转换再流程——二、三章先立两端，即 X 是什么、Y 是什么；四章给**映射规则 F**；五章给“执行与验收”——章首先给一段引题、随后总览表，Start、Step 1...N、End 各成一节，读写哪些产品数据随它所属的那一节就地成文。

2.2 组织原理

- **一份 SKILL 指南说明本任务这一步的 X→Y**：输入文档是**自变量 X**、输出文档是**函数值 Y**、转换规则是**把 X 映射成 Y 的规则**，执行步骤及检验是**怎么执行、怎么验收**。**这一步不等于整个需求-方案对**——阶段一只产出 Y 的较浅形态，阶段二才把 Y 补到最深，本指南只覆盖它自己那一步，见 §1.2。
- **概念跟文档走、原理跟规则走**：描述 X 的概念进第二章，描述 Y 的概念进第三章；解释“规则为何如此”的原理与其规则同章同节，不另设独立的“概念/原理”章。理由：在逐对生成里，所谓“概念”，如节点类型、框架 / 概要 / 定义判据，本身就是某侧文档的结构或规则，没有脱离文档而独立存在的概念层。
- **不设孤立的知识层三章、以设计动作序列为纲**：旧版 SKILL 指南的第三章即操作流程，按会话 Step 组织（Start / 1 / 2 / End / 反馈 / 写回），每节讲一个 Step 里 AI 与人怎么交互；本骨架改按**设计动作序列组织——即本任务的 Step 1...N**，见 §三.5。会话协议本身——反馈双轨状态机、git 基线、写回脚本、Start 与 End 两端的固定机制——属 91 附录A 运行时协议，SKILL 指南只引不重述；但 **Step 的划分与每步动作属本任务特有，要写**。
- **只引用、不搬运**：schema、运行时协议、方法论证各有单源，见 §四.1、§1.3；SKILL 指南写“本任务怎么用”，不复制权威正文。

2.3 输入/输出文档的树形结构图

每一份 SKILL 指南的**第二章与第三章**，都必须给出所涉产品数据的**树形结构图 + 逐级节点语义**——第二章管输入，给 **X 的树**；第三章管输出，给 **Y 的树**。

为什么必须给：二、三两章先立两端，即 X 树、Y 树；而第四、五两章的规则全都要指着这两棵树说话——承接规则要说“X 的叶落到 Y 的哪一级”，分解源判据要说“这个枝梢能分解出哪些叶”。**树不给出来，这些规则就没有可指的对象**。

图怎么画：

- 图内只写**节点名称**：纵向嵌套、逐级列出；级位编号 ①..., 按该文档的实际级数。节点意义另列成表，见下。
- 只画**本任务的任务视图**——本任务不涉及的层级不必展开；树权威 = 94, **不逐图注明**——文件头的自足声明已列它；不得另立一套树口径，见 §4.1。
- 输出横跨多条产品树的，各给一图；本任务的输出只装一棵树的，给一棵。
- 与上游同指的节点——叶由上游直接给出的步里整段搬入的那几级——**意义只写「同 X 树的同名节点」**，不重述它们是什么；它们是什么，第二章的树已说过。**该级的类型判据与形态身份不受此限**——正式 / 轻量、执行类 / 管理类的判据仍写在本表那一行 (§3.3)，枝梢与叶的身份同此。

图外给表：节点意义与节点**"实现规则"**都写进表里，**不占树位**。

- **两章同给第一张表**：级 | 节点 | 意义，节点意义即 94 树里的节点意义。
- **输出侧再加第二张表**：级 | 节点 | 实现规则——实现规则逐行取值，取四种来源，即承袭、本任务产出、AI 建议+人类确认、目标组织级，且"本任务如何处置"并入同一格，因为它与该级的来历同属本任务造 Y 的过程。两样分列：意义是 94 的树搬来的，实现规则是本任务的特化，同格并列会让人分不出哪句是树权威、哪句是这一步定的。
- **输入侧按需**：X 的各级怎么被本任务用，能并入第三章实现规则表的即并入；Y 里没有对应级的（X 比 Y 多出的那几级），另给一张 级 | 节点 | 处置。

图与表都带编号与标题：编号取「章-序」——章号加该章内的序，如 图 3-1、表 3-1、表 3-2；标题接在编号之后，合成一行加粗文字，紧贴在图或表之前，如 ****图 3-1: R 树（各业务域 · 业务运行职责视角）****。**本类文档全篇的图与表同此**，不限本章的树图与树表——第四章的判定表、第五章的总览表、受理门槛表与读写表都编号带题，正文要指某张表时用号指。

全链 SKILL 指南 biz/eng/nfr 均须遵守并回填。

2.4 正文写法（跨章通用）

本节规则对本类文档全篇一律有效。前三条是所有文档共有的底线，规则本身见 00-doc-conventions §八，此处只给 SKILL 指南上的执行口径；其余为本类文档特有。

一·每一段自治，不必回翻。一段读完就应当立得住，不靠跳到别处才懂。由此两条：**本文件内部的小节引用一律不挂**——读者就在同一份文件里，要什么往下翻即是，挂上去反而像"答案在别处"；**对外部权威源的引用越少越好**——它们的规则各有单源、已在文件头列明，正文只在"这一条不在此处展开"时才挂一次；**同一源在不同章的引用须指向它的不同部分，同一处规则不重复挂**。引用作主语谓语时，即「见 §X」充当主干，读者须离开当前句完成一次跳转，回来已丢掉前半句，一律改写成把话讲全。**分工声明不算引用**——「按 X 执行」「以 X 为准」「X 单源为」「本任务在此写它这一侧的特化」这类句子，说的是本文档与另一本文档把规则怎么分了，读者要的那个答案这里没有；它把读者推去别处，却不告诉他读到什么为止。判据：**删掉那处引用，本句是否仍完整可读；删不掉，就把话写全**。同理，删掉那句分工声明，本节是否仍完整可读——仍可读，就该删。

「固定机制按 X 执行」是这类句子里最坏的一种：它连规则名都不给，读者须先知道那份文档的全貌才敢动手。

二·不写元评论。 文档不谈自身——不写本文件的章结构为何如此、不重复文件头已声明的命名约定、不写本文件不重述什么或不另设什么、不写本文件与哪份文档并存或互为候选。判据：**这句话是在讲「这件事怎么做」，还是在讲「这份文件是怎么写的」。**

三·不写来历与待办。 正文只写现在是什么；来历归 Git，待办与挂账归 workflow。头注只记两样——创建日期，和至今经历了多少版本；修订日期、历次改了什么、谁改的都不写，一律归 Git。既有 wft / ort 指南章末的「变更记录」节是本骨架之前留下的，不在本规范之列。判据：**读者读完一段应能直接判断它现在有没有效力，不必先弄清它是哪一版改的。**

四·代号白名单。 正文只允许四个代号：**X** 为输入文档、**Y** 为输出文档，二者即本次映射的自变量与函数值；**R 树**、**P 树**。其余一律给出具体指代，缩写与编号同此——首次出现给全称，如「可交付成果分解结构（WBS）」：

代号类型	落法
层级序号 ① ② ③ ...	只出现在 树形结构图内 与 以级为标识的表的「级」列 ，规格见 §2.3； 散句、节题、表题，以及「级」列以外的单元格，一律写节点名 ；也不用「第 N 级」这类序数
需求-方案对序号 ① ② ③	按两端文档称呼，如「相关方需求 → 业务流程」「业务流程 → 系统需求」。同一文件里层级序号也用 ① 时尤其忌——一个符号两种含义，读者须先判断是哪种
产品数据编号 01~07	写本名，如「相关方需求」「职责视角业务流程」「产品开发视角业务流程」「系统需求框架」「系统需求概要」「平台构成框架」；编号只留给文件名、节号引用与链上坐标图
资产编号	不设 ——产品数据即资产，R 树与 P 树即资产，不另立编号
规范与文档编号 91 / 93 / 94 / 20 号	首次出现给文题，如「94 号文档《EOS 系统设计产品数据分解结构》」；其后可用编号简称
任务代号	按链上位置称呼，如「阶段二任务」「需求预处理链」「引擎能力设计任务」
需求类型代号	写全称，如「业务功能需求」「引擎能力功能需求」「非功能需求」
节点类型代号	写本名，如「业务流程概要节点」
类型代号 D / PCA	写本名，如「执行类」「管理类」
状态值、节点 ID、 字段名与字段值	保留原值 ——它们是要写进产品数据文件的字面量，不是叙述代号

「以级为标识的表」= 表身里靠圈号认级的那些表——第二章的输入侧处置表、第三章的级语义表与实现规则表、第四章按级列判定的转换表。在这些表里圈号只承担认级一件事，落在

「级」列或左右并排的对应格上；其余各列、各格里仍写节点名。「第 N 级」这类序数与圈号同罪——读者一样要回翻树图才数得出是哪一级。集合称谓不在此列：「末三级」「这三级」可以说，但要先交代它们含哪几级。

圈号成片，多半不是因为作者疏忽：树权威通篇以 ⑤ ⑥ ⑦ 说话，在它那里圈号就是级的本名。同一句话搬进 SKILL 指南就违规——SKILL 的散句里没有那棵树作上下文，读者须回翻第二章或第三章的树图才认得出。

五·文件头前言块。 # 标题 之下、第一章之前给一个引用块，含四行：本文件是什么、命名约定、自足声明、版本。不含与其他文档的关系陈述，也不含本文件的编写进度。链上坐标属第一章，不进文件头。

六·指代就地兑现。「这一对」「本对」「下一对」「该节点」「上述」这类只在上下文里成立的指代，读者必须回翻才知道指谁——尤其当同一文档里层级的量词不止一种时（对 / 阶段 / 级 / 树），「这一个」到底数的是哪一类要靠猜。**写具体：**能写两端就写两端，如「相关方需求 → 业务流程」；写不了就给一个自释的量词，如「本次转换」「这一步」。

七·用权威里已有的词，术语写全名。文档名、节点名、动作名一律取权威单源里已有的说法——产出、移交、合做、干枝、概要、定义；不为求顺口另造同义词。术语按本名写全，缩写首次出现给全称、其后可用——「末三级」要先交代为「业务文档、任务节点、用例场景」，没交代时写全名。判据：**这个词在 91 或 94 里找得到吗；找不到，那就是转述而不是引用。**本条也管给规则作结的短句——不写「这是××机制」这类自造的理由标签：它既不是解释，也没说清什么被启动、为什么这样就是机制，也查不到出处，读起来像理由而已。

八·一句一个主语，公式不作句子。一句话里主语不来回跳，也不用破折号、分号把几件事硬接成一句。无主语的公式单独成句不成立——「根→叶、各级只到说明」要写成「根到叶，各级只到说明」。判据：**这句念一遍，能说出是谁在做什么吗。**

九·只陈述，不对话、不抒情。正文陈述规则与事实。不用第二人称——写「人类确认」「人类裁决」，不写「要你确认」「由你定」；节内引题用陈述句——写「产出的形态是概要」，不写「为什么停在概要」；不用比喻与文学腔——不写「停得浅一点的样子」「写出这棵树的秩序」；不写与读者对象错位的表态句——操作规程里不写「取信于××——「...」」。

十·一处一说。同一件事只在一处说完整；树形结构图、图注、正文与条目之间不复述同一批事实。

十一·不写编排的说明。正文只给规则本身，不给规则的编排。**三不写——不汇总若干节合起来的样子（「哪几步并非每轮都走」「本任务的检验共三处」）；不作比较（「并非」「不再」「才是」「都在这段对话里完成」）；不作安置的理由（「所以只能自上而下逐级走」「所以它排在长干枝之后」）。**各步何时走、检验落在哪几处，就地写在它们所属的那一节；步与节的先后由步序与总览表承载。判据：**删掉这句话，执行者照章办事会不会做错——不会，它只是在讲编排。**与第二条同族：那条管文件自身的结构，这条管任务自身的编排。带这套味道的句子多半是后加的——

作者改动了编排，就用一句话把改动讲给读者听；这一句对第一次读的人来说是噪声，对读过旧版的人来说才是信息。

其余各节直接适用，不必另行特化。 00-doc-conventions §8.1 文档定位、§8.2 人称与视角、§8.3 规则表达硬度、§8.4 解释方式、§8.6 对比示例写法、§8.7 FAQ 与常见错误写法，以及 §8.8 的单句单意、慎用「进行」「加以」、避免重复表述、避免教科书式铺垫四项，在 SKILL 指南上照原意执行。上表与本节各条的写法优先于泛指的一般写法，凡未列出的按 00-doc-conventions §8 原意办。

三、各章编制规则

每章统一给：应含 / 禁含 / 规则来源。逐章规则随链上落地继续回填，带【待回填】处未定案。

3.1 第一章 目的与目标

本章四节，次序即下列应含项的次序：**1.1 主要目的 / 1.2 在设计链上的位置 / 1.3 主要目标 / 1.4 与上下游的衔接。**

应含：

- **主要目的：**一段话说明本任务要达成什么，写成引用块。**列在本章之首**——先把“这一步要达成什么”立住；坐标是给目的一定位置的，反过来读就得先记一串位置、再猜它要干什么。
- **在设计链上的位置：**本任务 = 哪个“需求-方案对”的**做哪几级**——a 框架设计、b 概要设计、c 详细定义，连写即合做（记号见 §1.2）；X、Y 各为哪份产品数据的哪个形态。给一张链上坐标图。**同节并含三样：形态的位置**——本任务交出的形态，以及该形态在一个需求-方案对里的身份：框架、概要、定义都是**方案的不同详细程度**，一个需求-方案对的两端是需求与方案两份产品数据，三者都不算对的一端（见 §1.2）；**本步停在或穿过某形态的理由**——合做的要写明哪几级一手做完、哪几级留给谁；**人类门禁位置**——本任务在哪几处交人类确认或裁决，以及节点状态由哪个状态值推进。
- **主要目标：**逐条列出本任务这次运行要产出的结果，写成编号列表。**所写范围覆盖本任务的职责面**——逐条与该章之后各节的产出对应。
- **与上下游的衔接：**X 从哪里来；下游由谁接，是阶段二执行者还是链尾接手方；检出缺陷时退回给谁、按什么条件退。写成一张表，必要时表下加注。

禁含：schema、规则、流程细节——这些属后章。**不设“负责 / 不负责”节。**

职责与边界为什么不各设一节：负责是主要目标的复述——逐条比对下来，负责的实质内容目标里全有，复述出来只是同一件事写两遍。**不负责是负责的补集，不可穷举**——按“链上有哪

些环"去列，环是无限的；退到"只列与本任务产出物直接相邻、容易被误认的那几处"，也仍是一条自证不全的清单。本任务不做事各归其位，散布在它实际发生的地方：X 侧归第二章的规则，Y 侧归第三章的边界，外环归第四章的衔接表与第五章各步的读写。

3.2 第二章 输入文档结构及规则

定位：X 的需求侧视图。每份 SKILL 指南只描述一份主输入文档；它是哪一份，由下面的"输入文档三类语义"决定。

输入文档三类语义：按本任务在对中的角色，X 属三类之一：

- **承接型：**X = 上一份产品数据的最深可用原子，即它的叶——它即本需求-方案对的需求。此处不用形态词概括：94 §1.1 明示「01 相关方需求不分形态」，其叶 = 需求条目；其余上游取其最深形态。94 §1.4 的承接规则一律以"叶"表述。本任务是新"需求-方案对"的阶段一，任务是承接；
- **深化型：**X = 本需求-方案对阶段一产出的 Y，即该对方案侧的较浅形态（框架或概要）、对内中间形态。本任务是同一需求-方案对的深化步，任务不是承接而是深化——把已承接语义从说明级写到详细定义。
- **组装型：**X = 上游产品数据，且其最深一级就是本案 Y 的最深一级——X 的末级节点整段成为 Y 的末级节点。本任务不做映射、不做分配，但要按属性在 Y 的树上逐级定位，逐级判定它是复用、改进还是新增。叶由上游直接给出的步属此类——相关方需求 → 业务流程，以及职责视角业务流程概要 → 产品开发视角业务流程概要。

三类在第二章标题下内容不同：承接型的 X 是上一份产品数据的叶，第二章标出这些叶在 X 树里的位置与归属；深化型的 X 就是本对较浅形态的那棵树，第二章给这棵树本身；组装型的 X 的最深一级即本案 Y 的最深一级，第二章写明末级节点各带哪些属性、以及这些属性各自定 Y 的哪一位。更上游文档不作第二章文档——只作覆盖检查锚点，经承接记录/追溯引用。

本章只立"X 是什么"：X 的树、各级语义，以及结构出处。凡"本任务拿 X 做什么"的话——收哪些、一次多少、读到瑕疵怎么办、不合规退回给谁——都是执行动作，归第五章的入口检验。理由有二：校验由"条件→判定→动作"三段构成，动作只在执行时有意义；且本规范对承接本就有同样的分工先例——承接规则立第四章、单承接检查归第五章，规则与检查不混在一章。

应含：

- **X 的树形结构图：**画出 X 的树，逐级写节点名称；图外给一张各级节点的意义表——X 的产出者对整棵树是同一个，写在树图之前的正文里说一次。规格同 §2.3。
- **X 的树形结构规则：**那些不进节点定义的结构性的规矩——属性随节点携带、不占级位；同一对象可对应多条上游需求——作意义表的注。
- **schema 引用：**只引 X 的文档结构的出处——91 §11 加该产品数据文件自身——不复述结构本身。

禁含：X schema 全量；本任务旧 Y 的存量，归第三章和第五章的修订流程；对 X 的检验与对 Y 的关口，一并在第五章。

3.3 第三章 输出文档结构及规则

定位：Y 的方案侧视图。

应含：

- **Y 的树形结构图：**画出 Y 的树，逐级写节点名称；图外给一张各级节点的意义表。规格同 §2.3。
- **节点类型判据：**引用 schema 出处，并给出本任务产出的节点类型判据——**正式/轻量、执行类/管理类两种必给**，各与它管的那一级同格，写在意义表里；「叶的独立判定」只在任务自造叶时给，叶由上游直接给出的步不设。
- **Y 树各节点实现规则：**图外再给一张表，逐级给出每一级的来历与处置——承袭、本任务产出、AI 建议+人类确认，还是目标组织级；**按级并进该表**、逐行成格的是**构建规则**（父节点归组）与**形态边界**（本任务交出哪个形态、该形态下要素写到什么深度）；**收敛自查**（"覆盖完整 + 兄弟互斥"）管的是整棵树、落不进任何一行，写成该表之下的独立一条；停在框架的另写明**枝梢停在哪个阶**（框架有阶，见 94-eos-sys-dev-dbs.md §1.3）。规格同 §2.3。
- **枝梢节点的双向物理意义：**承接判据，即该枝梢若是跨需求-方案对承接的落点，它承接哪类需求；分解源判据，即它能分解出哪些下一级节点，是移交下一形态 / 下游的规则接口。**只对自造枝梢或自造叶的任务有内容——叶由上游直接给出的步整项不设：**末三级与上游同指、整段搬入，既不承接也不分解，树图里只作指认。
- **要素清单：**本任务产出节点必须带齐的要素。
- **移交承诺：**交到下游前节点须达到的状态与完备程度，供 B 的第二章消费。

禁含：阶段二的执行内容，即叶节点与字段级详设；schema 全量，只引单源。

形态边界：树级语义取自 94-eos-sys-dev-dbs.md §1.3——**枝梢 = 框架的最深一级、叶 = 概要的末级**，即枝梢是叶的上一级；**框架有阶**，枝梢取哪一级由该次框架设计定。**停在框架的任务**，其枝梢内容可含"分解源判据的展开声明"——下一级清单 + 活动行为锚点，各项带内联稳定标签作追溯键；**展开项不是框架的树节点**，其身份在产出概要时转为叶。**本任务交出概要的，其叶已是树节点，不另设展开清单。**

叶由上游直接给出的步：业务文档、任务节点、用例场景三级由相关方需求直接给出；任务节点 = 枝梢，用例场景 = 叶。文档正文不占树位。

承接源只取上游叶：跨需求-方案对承接的**定义域只取上游叶**。94 §1.2 依据三写的是：**定义域取需求侧的原子，即叶——粗粒度节点不能被 1:1 分配**，它内部含着多条性质不同的需求，交出去后无法判定谁负责哪一部分；94 §1.4 的承接规则同此。**故枝梢不参与承接。**

系统需求框架的枝梢 = 用例场景功能：94 §1.4 的承接表把它写作 04→05 的下游落点，94 §2.3.1 的 SR-F 树里它正是枝梢。**功能表单不属于系统需求树——它是「用户操作页面视角」**

3.4 第四章 输入到输出的过程及规则

定位：X→Y 的映射与判定，即承接规则在本需求-方案对上的实例化。

应含：

- **转换过程：**X 走到 Y 的步骤顺序——分几步、每步做什么、每步在 Y 的哪一级上落，把随后各段规则串成一条路。**步骤止于 Y 落成；**一次运行的 Step 序列属第五章。**一节一步，标题写「第 N 步·动作名」：**动作名**动作领起、以它的对象收尾**，动作前后可带状语——来源、依据、范围或时点（「由...」「基于...」「为...」「...后」）；不带指代、不用「把」字句与条件分句——标题会被抽进目录单独读，读者从标题要能看出这一步做什么。
- **承接/分配规则：**X 的叶与节点如何落到 Y 的各级干枝/枝梢——**承接规则与基数引 94 §1.2~§1.4，不在此重述；**承载量超限则拆分节点；切不可走横切式承接或独立分支，本需求-方案对适用时。**只对承接型与深化型适用——**组装型的 X 最末一级就是本案 Y 的最深一级，叶由上游直接给出、整段搬入，不是承接，整项不设。
- **类型与要素生成判定：**从 X 内容推出 Y 节点类型/要素的判定规则。
- **修订转换规则：**已有 Y 时，新 X 如何并入——复用 → 改进 → 新增 的分流与关系检测。
- **对齐规则：**Y → 关联产品数据的判定动作，如 Q 判定——判定动作在此定义。本任务有旁支产品数据要对其时适用。
- **影响面复查：**对 Y 枝梢做 Extend/Split/Merge 改进时，对既有承接的复查规则。

禁含：一次运行的 Step 序列，即载入、反馈处理、写回、Start 与 End 两端，属第五章；**缺口的判定与处置，归第五章的关口；**关联产品数据自身的结构，归其数据文件；**第三章已给的判据本体与构建规则只引名，不复述本体。**

3.5 第五章 执行步骤及检验规则

定位：设计动作序列，本任务的检验就地成文于其中。

应含：

- **章首引题与总览表：**章首先给一段**引题**，随后即 **Step 布局总览表**。引题只写一次运行的起止——起于 Step Start、走 Step 1...N、收于 Step End。**各步何时走、检验落在哪几处，就地写在它们所属的那一节，引题不复述——**汇总一句「哪几步并非每轮都走」只是把下面各节又说一遍，且读着像在交代这套编排的来历。不逐行复述表，也不解释各列的列义（列义见 [00-doc-conventions §13.2](#)）。总览表的列定义与编号约束按 §13.2 / §13.3——全链指南共用同一张表的编法，不各写一套列；该表另在 SKILL 第三章复用，取列以 91 附录 A §A.2.4 为准。总览表之后不另设总览节，**Start、Step 1...N、End 各成一节紧接展开。**
- **Step Start：**标题写 Step Start · 动作名；本任务在入口这一侧的特化有两样。

- **受理门槛表**——什么样的 X 才归本任务处理：状态、范围、一次多少，不满足就**退出**；不合规**退回**给谁重切；受理之后的挂起与路由怎么走。**判定列按五类取值、全链同名同义**——**不可处理**（硬退出：状态不符、类型不匹配一类逻辑上不可能的条件）、**软提示**（交人类定夺，不强制退出）、**退回上游**（不合规的 X 交回上游重切）、**路由错误**（走错路径的 X 指回它该去的任务）、**无待处理对象**（兜底退出）。表里只列输入侧的合法状态——Y 侧退回态是三类入口之一（91 附录A §A.4），其门禁与路由归 Start，不入本表。
- **状态推进与退回**——**状态机图与写入分工并给**：主线、上游变更分流、下游退回三条路径与 91 附录A §A.6 的状态机——对应，缺一条路径的图会把合法的流转挡掉；并声明本任务写入哪几个状态、各在什么条件或时机下写入，由下游写入的那几个也点明。**两种退回各写去向**——X 有缺陷退回给 X 的上游，Y 被下游挑出则由本任务修订；**人类门禁**由总览表的「人类参与」列与状态推进节承载，不另设一条检查。
- **不写固定机制**——变更感知、入口的检测与优先序、退回门禁由 91 附录A 单源规定，指南不写、也不挂一句「按其执行」；本节只给本任务自己的动作，读者不跳转就读得出做什么。
- **Step 1...N，一节一个 Step**：标题写「Step N · 动作名」，名体例与第四章同，即动作领起、以它的对象收尾，动作前后可带状语，不带指代、不用「把」字句与条件分句。**每一节详细且自洽**——读完这一节就能照着跑，不必回翻别处：动作序（本步依次做哪几件）、触发（什么条件下走本步）、产出与去向（产出什么、谁接）、人类参与点（在哪停）、失败处置（卡住往哪走）。**Y 被下游退回的修订要有落点**——写清它落在哪一步、该步的触发怎么认领退回节点，以及它相对讨论确认步的位置；只写一句「走修订转换」而无落点，读者无处执行。**本任务自己的检验落到跑它的那一节**——关口、机械检查就地成文，不另设一节；跨章的判据（X→Y 的判定规则）只引第四章的规则名，不重述本体。**读写的旁支落进跑它的那一节**——本任务读哪些产品数据、写哪些，随它所属的那一步就地成文，不另设接口章。序列形如 选择与校验〔选定处理对象 → 受理门槛〕→ 生成〔设计循环 → 有改则自下而上调树至收敛〕→ 完备性/覆盖检查 → 讨论确认 → 入库。**Step 数随任务而异，属本任务特有**。不另起一套与 Step 平行的动作编号。
- **两处检验的判据本体**，落在跑它们的那一节里：
 - **单承接检查**，出口有二：程序违规回炉，或切不动的整体，**只对承接型适用**，叶由上游直接给出的步不设此项；
 - **承接覆盖检查**，式子按输入文档三类语义取值——承接型与深化型取「承接的 X 语义 $\subseteq$ 其分解产出之并」，组装型取两面，「X 的末级节点整段进入 Y、一个不少、同名同级」+「其文档序列兑现这批条目」；EOS 实例 = 闭合检查。**缺口在此成文**——哪一类缺口归本任务、锚在哪一级、候选去留谁裁、走什么回路；缺口不脑补，走补充需求材料回路。
- **Step End**：标题写 Step End · 动作名。写本任务在收尾这一侧的特化——**写回与状态推进、增量标记与自检清单**：写回按讨论结果落账、推进节点状态，规则就地成文；写回已随对话确认就地落盘、不在 End 收口的，End 只作本轮成果总结。增量标记引 91 附录A 的格式约定 +

本任务增量清单示例；自检按本任务流程分组逐项核对。三区模板的变量与结果行就地填。三区摘要输出这一固定机制由 91 附录A 单源规定，指南不写、也不挂一句「按其执行」。

- **运行时特化参数就地成文，落进它所属的那一节**：锚点需求、操作条目、推进目标、清除条件、级联触发条件属讨论确认那一步（91 附录A §A.5）——**讨论确认分几处，就按处各写它那一处的操作条目**；退回节点状态值属 Step Start 一侧，随状态推进写。其中**清除条件必须显式定义**——写明【已处理】的条目在何种新输入下回到待处理，不写成「由状态机定」。

禁含：EOS 会话运行时协议的复述与转引——反馈单轨与确认时点、git 基线、写回脚本、Start 与 End 两端的固定机制。这些由 91 附录A 单源规定，指南不写；**写一句「按 91 附录A 执行」同样禁止**——它连规则名都不给，读者须先知道那份文档的全貌才敢动手，而且它说的是两份文档怎么分工，不是任务怎么做。本任务自己的 Step 划分，以及 Start 与 End 两端各自的特化，不在此列。

四、跨文档引用与对接

4.1 结构与判据的出处

任何文档类型的**结构**只有一个权威家；SKILL 指南只引用、只写“本任务这一侧的规则”，不重复定义结构。若结构需演进，改单源 + 改受影响的侧规则，不改多份重复。

文档类型	schema 之家	SKILL 指南中的写法
产品数据文档 01~07	91 规范 §11 文档结构 + 各数据文件自身 + 00-doc-conventions	第二章/第三章引用结构，只写本任务需求侧/方案侧的规则
节点类型判据，如正式/轻量、执行类/管理类、叶的独立判定	91 规范 §二/§7~§9 相应节	第三章引用并给本任务的判定特化
关联产品数据，即 X、Y 之外的旁支读写对象	其数据文件自身 + 94-eos-sys-dev-dbs.md	第五章：随读它 / 写它的那一步就地成文
运行时协议	91 规范 附录A	只引用

树与 schema 分工：分解结构的权威在 94-eos-sys-dev-dbs.md，分解结构即树形与级语义；节点 schema 指节点类型/要素/字段/状态，之家仍为 91 §11 + 各产品数据文件自身 + 00-doc-conventions；**分解终止标准的家为通用规范 §2.2.2**，分解终止标准即叶的判据、“某棵树切到哪一级为止”，94 §1.2 依据一与 §1.3 两处都指此。94 定义“树长什么样”，不重述节点正文 schema，也不重述分解终止标准。

树权威与过渡期：产品数据**树权威** = 94-eos-sys-dev-dbs.md——各章树形与级语义只此一处定义，含**R / P 树所指**与树间对应。产品数据文件的「树画像」区里的跨文档产品树 R / P 两族，**不另设权威口径**，按 94 对齐落地；权威树演进时改 94 + 受影响的侧规则联动。**过渡**

期：94 尚未收口的章节，即 §2.4 的「模型」层、§2.5 配置方案、§2.6 前后端组件方案，暂以既有数据文件树画像为准，随 94 演进收口。至于树图的**编制要件**，即每份 SKILL 指南第二章/第三章必给什么、怎么给，见 §2.3。

判据归属分级，按可复用面定家：schema 单源之外，设计判据同样按**可复用面定家——跨任务复用的判据上移收敛到权威层，任务执行特化的判据留在该 SKILL 指南**，每条规则只有一个家、不双写。**树级判据归 94、分解终止标准归通用规范 §2.2.2、方法级判据归 20 号报告、结构判据归 91、对齐判据归 91 §5.3.1、过程判据留在本 SKILL 指南**：

判据族	家	SKILL 指南写法
树形/级语义判据 ——叶·枝梢·框架·概要·定义、成树三条件、承接规则与基数	94-eos-sys-dev-dbs.md §1.2~§1.4 ；94 为树权威，树级口径不另立别家	第二章/第三章/第四章引用；不重述树级定义与承接规则正文
分解终止标准 ，即叶的判据	通用规范 §2.2.2 ；94 §1.2 依据一与 §1.3 两处都指此	引用——"切到多细为止"的判据不在此重述
方法级判据——承接覆盖检查、复用→改进→新增	20 号报告	引用 + EOS 实例化，不重述原理
结构/领域判据——正式/轻量、执行类/管理类类型、输出文档模型、枝梢双向定义	91 规范 §二/§7~9 相应节	第三章 给出判据本体 （可压成一句话）+ 本任务判定动作； 不挂指针 ——判据本体就是这一步动手要用的东西，读者不必跳转，见 §2.4 规则一
对齐判据——对齐 Q1~Q3（引用 / 扩展 / 新建）、WBS 归组、一致性三类、缺口推断	91 规范 §5.3.1 持判据定义	第四章 引用 §5.3.1 + 判定动作；引用数据即在被对齐的那棵树里取
过程/执行判据——宿主匹配、边界匹配四标准、关系检测、来源类型、要素生成动作、闭合检查的 EOS 实例、受理门槛	该 SKILL 指南自身，第四章/第五章	就地成文——每段规则一个家 = 本任务

理由：20 号方法中规则 F 的定义权在人、存档于权威层；SKILL 指南不是 F 的再定义处，而是"某任务如何执行 F"的执行处——结构性判据属 F 的方案侧规则、对齐判据属被对齐那棵树上的规则，SKILL 指南执行；执行动作本身，即怎么判、怎么验，是任务特有，由 SKILL 指南自持。既消除与 91 的双写漂移，又不把 SKILL 指南掏空成纯指针。

Q 族分家：91 内有三族同名 Q，引用一律按全名勿裸写——**对齐 Q1~Q3**，家 91 §5.3.1，引用数据即被对齐的那棵树；**能力覆盖 Q1~Q4**，家 91 §4.4 扩展规则，引用数据即平台引擎的

能力清单；**22 分类 Q1~Q3+**，家 91 §九，引用数据即相关方非功能需求树。族索引见 91 §5.3.1。

缺口三族各归一处：上表「缺口推断」指的是 **P 组装那一类**——按构件产品数据清单做名称级比对，判据在 91 §5.3.1。另两类的判据不挂那里：**文档序列类**——闭合检查核对这条业务对象闭环是否走完、缺了哪份业务文档或哪个任务节点，就地成文于承接该闭环的任务第五章的关口；**引擎类**——用例场景逐一对照平台引擎，接不了的操作，就地成文于其归属任务的第五章的关口。

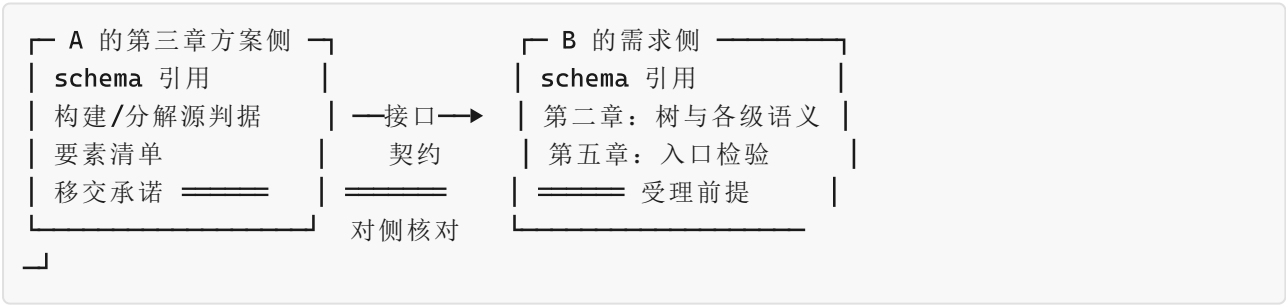
口径冲突：本规范内容与 91/20 号如有口径出入，以各自权威源为准并同步修订。

4.2 同一份文档的方案侧与需求侧

链上相邻任务共享**边界文档**：任务 A 输出它、任务 B 输入它。该文档被且只被描述两次，且两次天然不同：

- **A 的第三章，方案侧：**怎么产出合规文档 + **移交承诺**，即交到下游前达到的状态与完备程度，含枝梢分解源判据。
- **B 的第二章，需求侧：**怎么把它当 X 用 + **可承接对象**，即 X 的树与各级语义、本任务对每一级的处置；**受理前提**——受理门槛、读到瑕疵的处置、不合规退回——在 B 的第五章的入口检验。

A 的第三章的移交承诺与 B 的受理前提互为对侧——并排核对即本需求-方案对的接口契约校验；不吻合即对侧缺陷 → 退回 / 修订。受理前提横跨 B 的两章：可承接对象在第二章、门槛与检验在第五章，核对时两处并取。这一对侧关系就是防止"A 以为交全了、B 发现接不上"的机制，也就是 20 号方法反向迭代在文档体系上的落点。



核对方法 = 逐条映射，核对项为状态门槛、要素齐备、分解源判据、引擎权威、覆盖落账——边界分三类，各按此表逐条比对：**对内形态移交**，即同一需求-方案对内较浅形态交到较深形态，A 的第三章移交承诺 ↔ B 的第二章受理前提；**跨需求-方案对承接移交**，即上一份产品数据的叶落到下一对，A 的第三章 ↔ B 的第二章；**同对内的组装移交**，即叶由上游直接给出的步里，末三级整段搬入、干枝各长各的，前一步的第三章 ↔ 后一步的第二章。**一个任务位由两个并列任务分担两个视角时，各自与深化步构成一处对内形态移交**；只到概要、没有定义形态的那一侧，不进入下一对。

首条边界例外：链首那份产品数据即相关方需求，它由 ort 预处理链生成、并非 SKILL 指南，故其下游任务**没有对侧的一栏可核**——它的受理前提写在自身第五章的入口检验里，不进入并排核对。

4.3 输入文档的来源

本任务的需求树没有独立构建规则——它就是上一需求-方案对 / 上一任务的方案树，这正是 20 号报告 §2.4「需求树的来历」所说的。因此 B 的第二章不重述 X 的结构与构建，那是 A 的第三章的职责；只写“B 把 X 当 X 用”的受理与使用规则 + schema 引用。

覆盖逐对下传：“承接了的一定被实现”沿需求-方案对逐层演化为每个需求-方案对各自的检查内容——**叶由上游直接给出的步：**两侧的末三级是同一批，覆盖体现为“上游给的这批节点整段进入了 Y”；**承接步：**覆盖自查证明上一份产品数据已被这一份完整承接。每个需求-方案对的覆盖检查落在该任务第五章的关口。

链尾移交：设计线的链尾需求-方案对 = 系统需求 → 平台构成；链末份产品数据是 平台构成框架，见 94 §1.1。这一对只产出 Y 框架，末级 = 枝梢，枝梢带**分解源判据**；**该框架之下的叶由开发线阶段二执行**——开发线按分解源判据把枝梢节点分解为组件规格，设计线的产品数据链止于框架（20 号 §5.4）。该任务第三章移交承诺的对象 = **开发线首 skill**，设计线内无第三个需求侧；契约对侧在开发线规范——不需再造一份“开发需求”。**这一对的下游落点在 94 §1.4 标【待裁】**，即系统功能需求树的叶 IPO 功能 落到平台构成树哪一级。

两条“尾”要分清：**路径止点**——biz 路径在 **系统需求框架**止，biz 不入**平台构成**，见 91 §7.3，其后交构件开发流水线；**设计线链尾**——系统需求 → 平台构成。二者不是一回事，勿以路径止点代链尾。

五、与既有文档体系的关系

5.1 与 20 号方法的映射

SKILL 指南章节	20 号方法要素
一 目的与目标，链上坐标	需求-方案对与链定位：相对角色、树的三形态
二 输入文档结构及规则	X 侧之树与 X 侧分解规则，使用面
三 输出文档结构及规则	Y 侧之树与构建规则、枝梢分解源判据，即 §1.4 双向可用
四 输入到输出的过程及规则	承接规则：单承接，可 N:1 不可 1:N；复用→改进→新增。见 20 号 §2.5、本规范 §4.2

SKILL 指南章节	20 号方法要素
五 执行步骤及检验规则	预处理 + 正式设计循环、单承接检查、承接覆盖检查；旁支读写随步就地成文。见本规范 §三~§四、20 号 §4.5

5.2 与 91 规范的分工

- 91 规范 = EOS 领域设计模式 + 产品数据文档结构 §11 + Skill 运行时协议与格式 附录A 第三部分 + ort 预处理链 Skill 模板 附录A 第二部分，维持现状。
- 93 = SKILL 指南的**静态编制**规范。
- 91 附录A 第二部分「编写 Skill 文档」中的静态布局部分，将随本规范落地**迁出/改指 93**，待改造执行时改指；面向 ort 类的模板保留在 91。

5.3 与 SKILL 的关系

SKILL 指南是**权威源**；SKILL 是其**可执行提取**——SKILL 头部声明"设计依据 = SKILL 指南"，运行时细节引 91 附录A，脚本化数据访问/判据速查就地嵌入。本规范只约束 SKILL 指南静态编制；SKILL 的派生与同步规则由 **91 规范** 附录A + 各任务约定管辖，本规范不扩展。

六、编制工作流与检查清单

【待回填】链上首轮落地后成文。

6.1 编制/改造一份 SKILL 指南的步骤

【待回填】预期：① 认领本任务的 X/Y/阶段坐标，即 §三.1 那六条 → ② 写第二章/第三章，即需求侧/方案侧 + schema 引用 + 受理/移交 → ③ 写第四章/第五章，即转换规则与流程检验 → ④ 与相邻任务做接口契约核对，见 §四.2 → ⑤ SKILL 同步。

6.2 成稿自检

【待回填】预期四问：读第一章能说出「本步骤要达成什么、本任务在哪个需求-方案对、哪一侧、哪个形态、下游是谁」；读第二~四章能列出本需求-方案对的转换规则三块且知道谁定；读第五章能找到「承接设计动作」与「覆盖检查关口」；任取一段能读下去——不跳出当前句，也不为任何一个词去别处查它指什么。

6.3 待校准项

- "要素清单 vs schema"的分工写法。
- 移交承诺的成文化——拟以 02 边界两侧的接口对照校准。

附录A 空白骨架模板

【待回填】

附录B 编制实例

【待试写后回填：含第一章链上坐标，以及 02 边界文档的 A 的第三章移交承诺 × B 的第二章受理前提对照】